

Lingolali Project Bible

Product Vision

Lingolali is a multilingual language-learning platform built by extending the existing app, not by creating a new one. The platform's core strength is translation-first learning, and every new feature must sit on top of that foundation.

Scope and Product Goal

The goal is to evolve Lingolali into a complete learning system for any language, with flexible explanation languages, structured lessons, vocabulary practice, grammar study, quizzes, flashcards, and progress tracking.

Non-Negotiable Rules

- .Do not delete existing features -
- .Do not replace the current translation engine -
- .Do not rebuild the app from scratch -
- .Reuse the current Reader, Saved Words, Quiz, Flashcards, and TTS systems -
- Keep the current navigation and visual style unless a change is necessary for usability -
- .Extend the current architecture incrementally -
- .Keep the app lightweight and practical on free or low-cost hosting -
- Support Arabic as an explanation language, but do not limit the platform to Arabic only -
- .Support learning any language, not only Finnish -
- .Keep all curriculum content original and legally safe -

Existing Lingolali Features to Preserve ##

**Lingolali already has a strong base. These systems must remain active and
:connected**

- .Text translation into multiple target languages -**
- .Reader for reading and tapping words -**
- .Saved Words for storing vocabulary -**
- .Quiz for practice and review -**
- .Flashcards for memorization -**
- .Pronunciation and TTS -**
- .Multi-language interface support -**
- .Word saving, review, and editing -**
- .Existing UI patterns and navigation structure -**

New Main Navigation ##

:Keep the current app and add a Learn area

Text -

Reader -

Learn -

Saved -

Quiz -

.The Learn section is a learning center, not a separate product

Learning Platform Concept ##

:The user should be able to

.Choose a language to learn .1

.Choose a native or explanation language .2

.Study structured lessons .3

.Read examples .4

.Tap words for translation .5

.Save vocabulary .6

.Practice with quizzes .7

.Review with flashcards .8

.Track progress over time .9

Language Pack Architecture ##

**The platform should use modular Language Packs so it can teach any
.language**

:Example packs

Finnish -

English -

German -

Swedish -

French -

Spanish -

Arabic -

Kurdish -

:Each Language Pack should include
CEFR levels A0, A1, A2, B1, B2, C1, C2 -
Units -
Lessons -
Grammar modules -
Vocabulary modules -
Dialogues -
Exercises -
Review items -
Quizzes -
Progress tracking -

Learning Language and Explanation Language ##

:The user must be able to select
the language they want to learn -
the language they want the explanation in -

The explanation language should not be limited to Arabic. It can be any
.language already supported by Lingolali

Lesson Structure ##

:Each lesson should be structured and consistent
Lesson title -
Objective -
Vocabulary list -

Pronunciation notes -

Grammar explanation -

Dialogue -

Reading section -

Exercises -

Quiz -

Review -

Grammar Engine ##

:The grammar system should include

Clear explanation -

Examples -

Common mistakes -

Practice tasks -

Mini quiz -

Review path -

**Grammar should be simple and beginner-friendly, while still being useful for
.advanced learners**

Vocabulary Engine ##

:The vocabulary system should include

Target word -

Translation -

Pronunciation -

Audio -

Example sentence -

Save button -

Review state -

Difficulty or level tagging -

.Vocabulary should automatically connect to Saved Words and Flashcards

Reader Integration ##

.Lesson content should work with the existing Reader whenever possible

**Users should still be able to tap words and use the current translation popup
and save flow**

Quiz Integration ##

.Do not replace the current Quiz engine. Reuse it and expand it

:Support quiz modes such as

Lesson quiz -

Grammar quiz -

Vocabulary quiz -

Reading quiz -

Daily quiz -

Review quiz -

Flashcards Integration ##

Use the current Flashcards system and automatically generate cards from .lessons, vocabulary, and review items

Saved Words Integration ##

:Saved words should be grouped and organized into useful collections such as

Lesson words -

Grammar words -

Reading words -

Review words -

User favorites -

Progress and Gamification ##

:Add a simple motivation layer

Daily goal -

Weekly goal -

Streak -

XP -

Progress dashboard -

Completion indicators -

Video Learning Readiness ##

Prepare the architecture for future video learning without forcing it .immediately

:The system should be ready for

YouTube embeds -

Subtitle files SRT and VTT -

Local videos later -

Word tapping from subtitles -

Database and Data Model ##

:Add only the entities needed to support the new learning center

LanguagePack -

Level -

Unit -

Lesson -

GrammarItem -

VocabularyItem -

Dialogue -

Exercise -

ReviewItem -

Progress -

Streak -

QuizAttempt -

.Do not break existing database structures

.If changes are needed, make them carefully and incrementally

UI and UX Requirements ##

:The experience should stay close to Lingolali's current style

Clean -

Fast -

Mobile-first -

Easy to navigate -

Reader friendly -

Learning focused -

:Suggested learning flow

**Choose language → Choose explanation language → Pick level → Open lesson
→ Read → Practice → Quiz → Save words → Review later**

Performance Requirements ##

:The app must remain efficient on low-cost hosting

Lazy load lessons -

Cache translation results -

Avoid unnecessary API calls -

Keep assets lightweight -

Load only what the user needs -

Security Requirements ##

:Apply standard web security practices

Parameterized queries -

Input validation -

Output escaping -

Rate limiting -

Safe handling of user-generated content -

Avoid SQL injection and XSS -

Content Strategy ##

.All curriculum content must be original

.Do not copy text from other learning apps

Use the depth and structure of a professional language-learning product, but

.write the content from scratch

Roadmap ##

.Phase 1: Add the Learn area and language-pack structure

.Phase 2: Add lessons, grammar, vocabulary, and reading content

.Phase 3: Integrate Reader, Quiz, Saved Words, and Flashcards

.Phase 4: Add progress, streaks, and XP

.Phase 5: Prepare video learning and advanced features

Implementation Checklist ##

:Before making changes, confirm the following

.Existing Lingolali features remain intact -

.The translation engine remains the core system -

.The app is extended, not replaced -

.Arabic is supported as an explanation language -

.Any language can be taught through Language Packs -

.Quiz and Flashcards remain connected -

.The UI stays consistent with Lingolali -

.The app remains practical on low-cost hosting -

Final Direction ##

Build Lingolali as a long-term multilingual language-learning platform. Reuse the current foundation, add the new learning center, and keep every new .feature compatible with the existing platform architecture

Master AI Development Instructions ##

These instructions are for any developer, AI coding assistant, or development environment. Do not assume a specific framework, vendor, or hosting provider. .Work with the existing Lingolali codebase as it currently exists

Core engineering rules ###

- .Preserve existing user-facing behavior unless a change is explicitly required -**
- .Prefer extension over replacement -**
- .Prefer small safe increments over large rewrites -**
- Never break translation, Reader, Saved Words, Quiz, or Flashcards while -**
 - .adding learning features**
- Keep all new systems modular so they can be reused for more languages -**
 - .later**
- Keep the architecture portable across hosting providers and development -**
 - .tools**
- .Support future AI provider changes without redesigning the whole app -**
- .Optimize for readability, maintainability, and low operational cost -**

Implementation priorities ###

- .Keep the current app stable .1**
- .Add the Learn center .2**

- .Add multilingual Language Packs .3**
- .Add course content structure .4**
- .Connect Reader, Quiz, Saved Words, and Flashcards .5**
- .Add progress and gamification .6**
- .Prepare video learning and future AI features .7**

What not to do ###

- .Do not rebuild the whole platform -**
- .Do not hardcode the app to only one language -**
- .Do not hardcode Arabic as the only explanation language -**
- .Do not remove current features just to make room for new ones -**
- .Do not make the UI heavier than necessary -**
- .Do not copy lessons or content from other products -**

Complete Database Design ##

The data model should support a scalable multilingual learning platform. The exact implementation can vary by stack, but the logical entities should remain .consistent

Core entities ###

User -

LanguagePack -

Language -

Level -

Unit -

Lesson -

LessonSection -

GrammarItem -

VocabularyItem -

Dialogue -

Exercise -

Quiz -

QuizQuestion -

QuizAttempt -

SavedWord -

Flashcard -

ReviewSession -

Progress -

Streak -

XPEvent -

MediaAsset -

Suggested relationships ###

.A LanguagePack contains one Language -

.A LanguagePack contains many Levels -

.A Level contains many Units -

.A Unit contains many Lessons -

.A Lesson contains many LessonSections -

**A Lesson may reference GrammarItems, VocabularyItems, Dialogues, -
Exercises, and MediaAssets**

- .SavedWords may be linked to VocabularyItems or Reader selections -**
- Flashcards may be generated from VocabularyItems, LessonSections, or Quiz -**
 - .mistakes**
- .Progress tracks completion by user, language, level, unit, and lesson -**
- .QuizAttempts store history and scoring -**
- .Streak and XP are derived from activity events -**

Data principles ###

- .Keep entities normalized enough to avoid duplication -**
- Store lesson content in a structured form, not as one large blob whenever -**
 - .possible**
- Use a schema that can be extended by adding new lesson types and media -**
 - .types later**
- .Maintain backward compatibility with existing data -**

Learning Engine Specification ##

The learning engine is the heart of the new platform area. It should organize educational content into a navigable hierarchy and connect it to the current
.translation and review tools

Learning hierarchy ###

LanguagePack → Level → Unit → Lesson → Section → Activity → Review

Lesson behavior ###

:A lesson should
,explain one focused topic -

- ,provide target vocabulary -**
- ,provide at least one dialogue or reading sample -**
- ,include grammar clarification -**
- ,offer exercises -**
- ,end with a quiz -**
- .create review items for later practice -**

Level design ###

- :Use CEFR-like levels**
- A0: absolute beginner -**
- A1: simple everyday language -**
- A2: basic functional communication -**
- B1: independent user -**
- B2: upper-intermediate practical communication -**
- C1: advanced comprehension and production -**
- C2: near-native fluency support -**

Progress logic ###

- :Progress should be visible at multiple levels**
- ,lesson completion -**
- ,unit completion -**
- ,level completion -**
- ,streak maintenance -**
- ,vocabulary retention -**
- ,quiz performance -**

.review accuracy -

Grammar Framework ##

.Grammar content must be simple, structured, and reusable across languages

Grammar lesson template ###

Topic name .1

When it is used .2

Simple explanation .3

Examples in the target language .4

Translation or explanation language support .5

Common mistakes .6

Practice exercises .7

Mini quiz .8

Review suggestions .9

Grammar quality rules ###

.Keep explanations short and clear -

.Adapt complexity to level -

.Use examples from daily life whenever possible -

.Avoid jargon unless it is necessary and explained -

.Connect grammar examples to vocabulary already taught in the platform -

Vocabulary Framework ##

.Vocabulary should be tightly integrated with the rest of the app

Vocabulary item template ###

target word or phrase -

translation -

part of speech or type -

pronunciation -

audio if available -

example sentence -

level tag -

lesson link -

save action -

review status -

Vocabulary behavior ###

**Clicking or tapping a vocabulary item should expose the existing translation -
and save flow**

.Vocabulary items should automatically feed flashcards -

.Vocabulary items should support spaced review -

.Vocabulary items should appear in lesson summaries and review sections -

Reader Integration Rules ##

.The existing Reader is a major asset and should remain central

Required behavior ###

**Lesson reading content should open in the existing Reader whenever -
possible**

.Tapping a word must continue to use the current translation popup behavior -

.Saved Words must still work from the Reader -

.Reader-selected words can be promoted into lesson vocabulary -

.Reading content should feed quiz and review generation -

Quiz Framework Expansion ##

.The current quiz engine should be expanded, not replaced

Quiz types ###

Lesson quiz -

Grammar quiz -

Vocabulary quiz -

Reading quiz -

Review quiz -

Daily quiz -

Challenge quiz -

Quiz design goals ###

.Quick feedback -

.Clear scoring -

.Wrong-answer review -

.Spaced repetition integration -

.Mobile-friendly interactions -

.Difficulty scaling with level -

Flashcards and Review System ##

.Flashcards should become an automatic output of the learning system

Flashcard sources ###

Vocabulary items -

Lesson mistakes -

Quiz wrong answers -

Reader-saved words -

Grammar examples -

Review behavior ###

.Show weaker items more often -

.Prioritize recent mistakes -

.Support quick review sessions -

.Keep review sessions short on mobile -

Saved Words Architecture ##

.Saved words should become a structured library instead of a flat list

Suggested collections ###

All saved words -

Lesson words -

Grammar words -

Reading words -

Review words -

Favorites -

Manually added words -

Saved word metadata ###

source language -

target translation language -

where it came from -

whether it needs review -

whether the user edited it -

how many times it was missed -

how many times it was answered correctly -

Video Learning Readiness ##

**The platform should be prepared for video learning without forcing all
.implementation immediately**

Video features to support later ###

YouTube embed playback -

Subtitle display and parsing -

Clicking subtitle words to translate them -

Subtitle-based flashcard generation -

Subtitle-based review mode -

Support for imported SRT and VTT files -

Important design principle ###

Video learning must reuse the same translation, save, quiz, and flashcard systems rather than creating a separate isolated experience

UX and Navigation Details ##

.The learning flow should be simple and intuitive

Suggested user flow ###

.Open Lingolali .1

.Choose Learn .2

.Select a language pack .3

.Select explanation language .4

.Select level .5

.Open a lesson .6

.Read or watch .7

.Tap words .8

.Save vocabulary .9

.Practice quizzes .10

.Review flashcards .11

.Track progress .12

UX principles ###

,minimal friction -

,visible progress -

,low cognitive load -

,mobile-first design -

,fast loading -

.consistent visual language -

Content Authoring System ##

**The app should support future content creation without code changes for every
.lesson**

Authoring goals ###

.Add lessons without rewriting core logic -

.Add new Language Packs without redesign -

.Keep curriculum structure consistent -

.Allow future AI-assisted content creation with guardrails -

Authoring templates ###

**Every lesson should follow a predictable content template so that the UI and
.review systems can render it reliably**

Admin and Management Needs ##

:A future admin area should be able to manage

Language Packs -

Levels -

Units -

Lessons -

Grammar items -

Vocabulary items -

Exercises -

Media assets -

Review prompts -

Quiz templates -

Published status -

Future AI Features ##

The architecture should allow future AI-powered tools without rewriting the .platform

Possible future AI additions ###

AI tutor -

AI conversation practice -

pronunciation feedback -

reading comprehension help -

question generation -

automatic lesson drafting -

automatic quiz generation -

automatic review generation -

AI provider principle ###

The system should not be locked to one AI vendor. The architecture should .make it possible to switch or expand providers later

Security and Reliability ##

.All future work must remain safe and robust

Security principles ###

- .Validate all user input -**
- .Escape all user-generated output where needed -**
- .Use parameterized database access -**
- .Protect high-risk actions -**
- .Rate-limit expensive operations -**
- .Avoid cross-site scripting and injection vulnerabilities -**

Reliability principles ###

- .Handle missing content gracefully -**
- .Avoid breaking the current app if a new module fails -**
- .Cache expensive translations and lookups -**
- .Support partial loading and lazy loading -**

Final Master Prompt Statement ##

The job of the developer or AI assistant is to extend Lingolali into a scalable multilingual learning platform while preserving the current experience. The current translation-first system is the foundation. Everything new must be built .on top of it, not instead of it

Final Acceptance Criteria ##

:The project is successful only if all of the following remain true

- .Lingolali still works as a translation-first app -**
- .The current Reader still works -**
- .Saved Words still works -**
- .Quiz still works -**
- .Flashcards still works -**
- .New learning features work for any language -**
- .Arabic is supported as an explanation language -**
- .The app stays lightweight and practical -**
- .The platform can grow later without major rewrites -**